Avobl — Project Plan

A free, local AI assistant: chat, images and form-filling today. Next: a VigilX widget, a browser extension, and camera insights.

1 · Your PC: 8 GB RAM, Ryzen 5 8500G, no separate GPU

What 8 GB means
Windows shows ~7.1 GB because the built-in graphics reserves some. Windows + browser use ~3–4 GB, so the AI gets **~3.5 GB**: room for **one 3–4B model at a time**.

What works / doesn't
Yes: 1–4B models at ~10–14 tokens/s
No: 7B models: 0.5 tokens/s (swapping to disk)
No: two models loaded together: they push each other out (~10–20 s reload)

Best upgrade
Add RAM to 16 GB (2×8 GB DDR5, dual channel). That's cheap and the single biggest speed gain: two models stay loaded, and 7–8B models become usable.

2 · Why it is slow now, and the fixes

Cause (measured)	Fix
qwen3:4b is the thinking build: 3 min and ~5,500 chars of reasoning to fill 4 fields	Switch to **qwen3:4b-instruct-2507**: same skills, no thinking, still has tool calling
Runs 100% on CPU (graphics chip unused)	Try Ollama's Vulkan GPU mode; small gain, since the iGPU shares the same RAM
Switching text ↔ vision model reloads it (8 GB fits only one)	Fewer switches: route smartly, group camera jobs, keep one model warm

3 · Models: route each task to the right one (like Perplexity, but one at a time)

Role	Free local model	Size	Used for
Main	qwen3:4b-instruct-2507	2.5 GB	Chat, form-fill (tool calls), questions about a page
Fast	qwen3:1.7b	1.4 GB	Short, quick answers
Vision	qwen2.5vl:3b (have it)	3.2 GB	Image insights, reading text in images
Pre-check	moondream	1.7 GB	Cheap "anything happening?" check on camera snapshots
Optional cloud	Groq / Gemini / OpenRouter free tiers	—	Fast ordinary chat **only if you allow it**. Camera images never leave your PC

4 · What we build: one core, three ways to use it

Avobl Widget (VigilX)
One <script> tag adds a round pill. Clicking it opens a fixed-width side panel. It reads the page (text, tables, form fields) and fills forms after you click **Apply**. It never submits on its own.

Browser extension
The same widget code in a Chrome extension (Chrome's side panel), so Avobl works on any website. Its UI is isolated in a Shadow DOM, so site styles can't break it.

Avobl Watch (surveillance)
New snapshot every 10 min → change check → quick look (moondream) → full analysis only if needed (qwen2.5vl + your rules) → save insight → alert. Ask: "what happened at gate 1 from 2–4 pm?"

Capacity on 8 GB: a full analysis takes ~60–90 s, so about 6–10 cameras at one snapshot each per 10 min (more with the change check). **Training:** start with rules plus a few labelled example images, measure accuracy on your own footage, and fine-tune a small YOLO detector (free Colab GPU) only if needed.

5 · Phases

#	Phase	Done when
0	Foundations: git, faster models, model router, image-insight instructions	Form-fill answers in under 15 s
1	Core API: router + providers, settings, API keys, CORS	App and widget share one API
2	Widget: pill + side panel, page reader, form filler with preview	Works on a VigilX-style demo page
3	VigilX integration	Pill live inside VigilX
4	Browser extension (Manifest V3, side panel)	Same panel on any site
5	Avobl Watch: watcher, pipeline, database, alerts, timeline	Insights saved automatically every 10 min
6	Accuracy: rules screen, example images, test set, optional YOLO	Accuracy measured on your footage

6 · Needed from you before phases 3 and 5

1. VigilX tech stack (frontend and backend), and whether it's online or on a local network **2.** Where Avobl will run: if VigilX is online, other people's browsers can't reach Ollama on this laptop **3.** Where snapshots are saved, and how many cameras **4.** The first 3–5 things the AI should detect **5.** May ordinary chats use free cloud models?